

数据结构与算法 实验报告

第二次



姓名 陈宇昂

班级 数学强基 2202 班

学号 2224110096

电话 **

Email yuang.chen2024@outlook.com

日期 2023-12-22

目录

任务 1: 实现线性表的基本操作	2
一、 题目	2
二、 算法设计	2
三、 主干代码说明	2
(一) 顺序存储实现线性表	2
(二) 链式存储实现线性表	6
四、 运行结果展示	10
五、 总结与收获	11
任务 2: 栈	12
子任务一:	12
一、 题目: 两种解决 Hanoi 塔问题的方法	12
二、 算法设计	12
(一) 递归方法	12
(二) 非递归方法	12
三、 主干代码说明	12
四、 运行结果展示	15
五、 总结与收获	15
子任务二:	16
一、 题目: 非递归快速排序	16
二、 数据结构设计	16
三、 算法设计	16
四、 主干代码说明	16
五、 运行结果展示	18
六、 总结与收获	18
子任务三:	19
一、 题目: 双端顺序栈	19
二、 数据结构设计	19
三、 算法设计	19
四、 主干代码说明	19
五、 运行结果展示	22
六、 时间复杂度分析	22
七、 总结与收获	22
任务 3: 基数排序	23
一、 题目	23
二、 数据结构设计	23
三、 算法设计	23
四、 主干代码说明	23
五、 运行结果展示	27
六、 总结与收获	28

任务 1：实现线性表的基本操作

一、题目

在本次实验中，主要完成的任务是：

1、基于线性表的两种存储结构（即顺序存储和链式存储）编程实现以下功能：

InitList(&L):初始化表。构造一个空的线性表。

DestroyList(&L):销毁表。销毁线性表L并释放线性表所占存储空间。

Length(L):求表长。返回线性表L的长度，即线性表L中元素的个数。

Empty(L):判空。若L为空表则返回True, 否则返回False。

PrintList(L):输出表。将线性表的元素依次输出。

LocateList(L, e):按值查找操作，查找元素e在表L中的位置。

GetList(L, i):按位查找操作，查找第i个位置的元素的值。

ListInsert(&L, i, e):在表L的第i个位置插入元素e。

ListDelete(&L, i, &e):删除表L的第i个位置的元素，并用e返回删除元素的值。

2、要求对于每一种操作进行算法设计描述、实现描述及测试说明（须有程序运行截图）。

3、比较并说明上述操作在顺序存储和链式存储下的异同。

二、算法设计

算法设计细节部分与主干代码说明一起说明。

由于 Java 不直接支持引用参数，所以我将函数的参数进行了改进。

顺序存储：以数组作为数据存储的方式，此代码包含了错误处理，比如插入或删除操作越界，和查找为空的情况。

链式存储：以链表作为数据存储的方式。链表的操作主要涉及对节点的引用和解引用，以及遍历链表。插入和删除操作需要找到目标位置的前驱节点和当前节点，然后插入新的节点或断开对当前节点的引用。

三、主干代码说明

（一）顺序存储实现线性表

1、InitList

```
1 package task1;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 2个用法
7 public class AList<E> {
8     1个用法
9     private int defaultSize = 10; // 默认容量
10    13个用法
11    private E[] list; // 数组，用于存放元素
12    15个用法
13    private int size = 0; // 当前表长度
14
15    // 初始化线性表
16    0个用法
17    public void InitList() {
18        this.list = (E[]) new Object[defaultSize];
19        this.size = 0;
20    }
21
22    1个用法
23    public void InitList(int len) {
24        this.list = (E[]) new Object[len];
25        this.size = 0;
26    }
27 }
```

初始化操作，构造数组变量 `list`，然后创建给定大小（此处为 20）的数组（此处设置默认大小 `defaultSize` 为 10）来存储线性表的元素，同时用 `size` 记录当前线性表的长度。

2、DestroyList

```
15 // 销毁线性表
    1个用法
16 public void DestroyList() {
17     list = null;
18     size = 0;
19 }
```

销毁操作，我们只需将我们的数组变量设为 `null`，并将线性表长度重置为 0 即可。

3、Length

```
21 // 获取线性表长度
    2个用法
22 public int Length() {
23     return size;
24 }
```

返回线性表的当前长度，直接返回我们记录的当前长度数值 `size`。

4、Empty

```
26 // 判断线性表是否为空
    2个用法
27 public boolean Empty() {
28     return size == 0;
29 }
```

判断线性表是否为空，只需看线性表长度是否为 0，于是比较 `size` 与 0 的大小即可。

5、PrintList

```
31 // 打印表中元素
    3个用法
32 public void PrintList() {
33     for(int i = 0; i < size; i++) {
34         System.out.print(list[i]);
35         System.out.print(" ");
36     }
37     System.out.println();
38 }
```

通过 `for` 循环遍历并打印每个元素。

6、LocateList

```
47 // 按值查找元素位置
    3个用法
48 public List<Integer> LocateList(E e) {
49     List<Integer> positions = new ArrayList<>();
50     for (int i = 0; i < size; i++) {
51         if (list[i].equals(e)) {
52             positions.add(i);
53         }
54     }
55     return positions;
56 }
```

①创建一个空的数组 `positions` 用于存放找到的索引。

②遍历数组 `list` 以找到所有值等于指定元素的元素。我们将数组的长度用作循环的结束条件，并在循环的每次迭代中检查当前元素是否等于我们正在查找的元素。若当前元素等于我们正在查找的元素，我们将其索引添加到我们在第一步中创建的索引列表中。

③在数组遍历结束后，我们返回索引数组。如果该数组为空（即我们在数组中没有找到指定的元素），则返回的列表中不会有任何元素。

7、GetList

```
50 // 按位查找元素
    2个用法
51 public E GetList(int i) {
52     if(i >= 0 && i < size) {
53         return list[i];
54     }else {
55         System.out.println("查找位置无效");
56         return null;
57     }
58 }
```

按位查找，返回索引 *i* 处的元素。先检查 *i* 是否在合理范围内（0 到 *size*-1），如果在范围内则返回该处的元素，否则提示错误并返回 *null*

8、ListInsert

```
60 // 在指定位置插入元素
    2个用法
61 public void ListInsert(int i, E e) {
62     if(i < 0 || i > size) {
63         System.out.println("插入位置无效");
64         return;
65     }
66     if(size == list.length) {
67         System.out.println("表已满，无法插入");
68         return;
69     }
70     for(int j = size; j > i; j--) {
71         list[j] = list[j-1];
72     }
73     list[i] = e;
74     size++;
75 }
```

插入元素，首先检查索引 *i* 是否在范围内（0 到 *size*），如果超出范围提示错误并返回。然后从 *size* 开始，将每个元素向后移一位，直到索引 *i* 的位置，将 *e* 插入到 *i* 的位置，然后增加 *size*。

9、ListDelete

```
77 // 删除指定位置元素
    1个用法
78 public E listDelete(int i) {
79     if(i < 0 || i >= size) {
80         System.out.println("删除位置无效");
81         return null;
82     }
83     E deletedElement = list[i];
84     for(int j = i; j < size - 1; j++) {
85         list[j] = list[j+1];
86     }
87     size--;
88     return deletedElement;
89 }
90 }
```

删除元素，首先检查索引 *i* 是否在范围内（0 到 *size*-1），如果超出范围提示错误并返回 *null*。然后保存索引 *i* 的元素 *e*，从 *i* 开始，将每个元素向前移一位，直到最后，最后减少 *size*，然后返回 *e*。

10、测试代码（TestAList）

```
1 package task1;
2
3 public class TestAList {
4     public static void main(String[] args) {
5         AList<Integer> myList = new AList<>();
6         myList.InitList( len: 20);
7
8         // 检测初始化是否成功
9         System.out.println("初始的线性表状态: ");
10        // 测试线性表是否为空
11        System.out.println("线性表表是否为空? " + myList.Empty());
12        // 检测线性表长度
13        System.out.println("线性表长为: " + myList.Length());
14
15        // 测试插入函数（线性表从0开始）
16        for(int i = 0; i < 9; i++) {
17            myList.ListInsert(i, e: i+1);
18        }
19        myList.ListInsert(i: 9, e: 1);
20        myList.ListInsert(i: 10, e: 1);
21
22        // 打印表的长度
23        System.out.println("插入元素之后, 线性表长为: " + myList.Length());
24
25        // 打印表中元素
26        System.out.println("线性表中的元素为: ");
27        myList.PrintList();
28
29        // 测试按值查找函数, 查找值为5的元素位置
30        System.out.println("值为5的元素的位置为: " + myList.LocateList(e: 5));
31        System.out.println("值为1的元素的位置为: " + myList.LocateList(e: 1));
32        System.out.println("值为15的元素的位置为: " + myList.LocateList(e: 15));
33
34        // 测试按位查找函数, 查找位置为7的元素值
35        System.out.println("处于7号位的元素值为: " + myList.GetList(i: 7));
36        System.out.println("处于15号位的元素值为: " + myList.GetList(i: 15));
37
38        // 测试插入函数, 位置为5插入值为10的元素
39        myList.ListInsert(i: 5, e: 10);
40        System.out.println("在5号位后插入一个值为10的元素, 线性表变为: ");
41        myList.PrintList();
42
43        // 测试删除函数, 删除位置为8的元素
44        System.out.println("8号位元素的值为: " + myList.listDelete(i: 8));
45        System.out.println("删去8号位之后, 线性表变为: ");
46        myList.PrintList();
47
48        // 销毁表
49        myList.DestroyList();
50        System.out.println("线性表在销毁后是否为空: " + myList.Empty());
51    }
52 }
```

(二) 链式存储实现线性表

1、InitList

```

1  package task1;
2
3  import java.util.*;
   2 个用法
4  public class LList<E> {
   12 个用法
5      Node<E> head;
   9 个用法
6      int size;
7
8      // 初始化链表
   1 个用法
9      public LList() {
10         head = null;
11         size = 0;
12     }

```

初始化操作，我们创建一个空节点来作为链表的头节点，同时设置一个 size 变量记录当前链表的长度（即元素数）

2、DestroyList

```

14      // 销毁链表
   1 个用法
15      public void DestroyList() {
16         head = null;
17         size = 0;
18     }

```

销毁操作，我们将头节点设为 null，并将链表长度重置为 0 即可。

3、Length

```

20      // 返回链表长度
   1 个用法
21      public int Length() {
22         return size;
23     }

```

返回链表的当前长度，直接返回我们记录的当前长度数值

4、Empty

```

25      // 判断链表是否为空
   2 个用法
26      public boolean Empty() {
27         return size == 0;
28     }

```

判断链表是否为空，只需看链表长度是否为 0

5、PrintList

```

30      // 输出链表元素
   2 个用法
31      public void PrintList() {
32         Node<E> temp = head;
33         while (temp != null) {
34             System.out.print(temp.data);
35             System.out.print(" ");
36             temp = temp.next;
37         }
38         System.out.println();
39     }

```

通过循环链表并打印每个节点的数据。

6、LocateList

```
41 // 按值查找元素位置
42 // 4个用法
43 public List<Integer> LocateList(E e) {
44     List<Integer> result = new ArrayList<>();
45     Node<E> temp = head;
46     int index = 0;
47     while (temp != null) {
48         if (temp.data.equals(e)) {
49             result.add(index);
50         }
51         temp = temp.next;
52         index++;
53     }
54     return result;
55 }
```

(此函数规避了多个元素的值相同只输出第一个位置的情况)

初始化:

- ① 创建一个空列表 `result` 来存储找到的索引。
- ② 设定一个存储当前索引的变量 `index`。
- ③ 设定一个节点 `temp` 用来遍历，初始指向链表头部。

开始遍历链表:

① 检查当前 `temp` 节点的元素值是否等于目标值，如果相等，则将当前的 `index` 加入到结果列表 `result` 中。

② `temp` 移动到下一个节点，同时将 `index` 增加 1。

遍历结束后，返回结果列表 `result`，它包含了所有等于目标值的元素位置。

7、GetList

```
56 // 按位查找元素
57 // 1个用法
58 public E GetList(int i) {
59     if (i < 0 || i >= size) {
60         throw new IndexOutOfBoundsException("索引超出范围!");
61     }
62     Node<E> temp = head;
63     for (int j = 0; j < i; j++) {
64         temp = temp.next;
65     }
66     return temp.data;
67 }
```

按位查找，返回索引 `i` 处的元素。先检查 `i` 是否在合理范围内（0 到 `size-1`），如果在范围内则循环到该位置并返回该处的元素，否则提示错误。

8、ListInsert

```
68      // 在第1个位置插入元素e
69      5个用法
70      public void ListInsert(int i, E e) {
71          if (i < 0 || i > size) {
72              throw new IndexOutOfBoundsException("索引超出范围!");
73          }
74          Node<E> newNode = new Node<>(e);
75          if (i == 0) {
76              newNode.next = head;
77              head = newNode;
78          } else {
79              Node<E> prev = head;
80              for (int j = 0; j < i - 1; j++) {
81                  prev = prev.next;
82              }
83              newNode.next = prev.next;
84              prev.next = newNode;
85          }
86          size++;
87      }
```

插入元素，首先检查索引 i 是否在范围内（0 到 $size$ ），如果超出范围提示错误并返回。然后创建一个新的节点和 e ，并找到索引 i 的前一个位置的节点，将新节点插入到前一个节点和当前节点之间，然后增加 $size$ 。

9、ListDelete

```
88      // 删除第1个位置的元素，并返回其值
89      1个用法
90      public E ListDelete(int i) {
91          if (i < 0 || i >= size) {
92              throw new IndexOutOfBoundsException("索引超出范围!");
93          }
94          Node<E> removedNode;
95          if (i == 0) {
96              removedNode = head;
97              head = head.next;
98          } else {
99              Node<E> prev = head;
100              for (int j = 0; j < i - 1; j++) {
101                  prev = prev.next;
102              }
103              removedNode = prev.next;
104              prev.next = removedNode.next;
105          }
106          size--;
107          return removedNode.data;
108      }
```

删除元素，首先检查索引 i 是否在范围内（0 到 $size-1$ ），如果超出范围提示错误并返回 $null$ 。然后找到索引 i 的前一个位置的节点和当前节点，将前一个节点的 $next$ 指向当前节点的 $next$ ，从而断开对当前节点的引用，然后减少 $size$ ，然后返回当前节点的数据。

10、测试代码（TestLList）

```

1  package task1;
2
3  import java.util.List;
4
5  public class TestLList {
6      public static void main(String[] args) {
7          LList<Integer> list = new LList<>();
8
9          // 测试初始化
10         System.out.println("初始化之后表是否为空? " + list.Empty());
11
12         // 测试插入
13         list.ListInsert(0, 1);
14         list.ListInsert(1, 1);
15         list.ListInsert(2, 2);
16         list.ListInsert(3, 3);
17         list.ListInsert(4, 2);
18
19         System.out.println("在插入之后, 线性表为: ");
20         list.PrintList();
21
22         // 测试长度
23         System.out.println("线性表长度为: " + list.Length());
24
25         // 测试按值查找
26         List<Integer> loc = list.LocateList(2);
27         System.out.println("线性表中值为2的元素是: " + loc);
28         List<Integer> loc1 = list.LocateList(1);
29         System.out.println("线性表中值为1的元素是: " + loc1);
30         List<Integer> loc2 = list.LocateList(3);
31         System.out.println("线性表中值为3的元素是: " + loc2);
32         List<Integer> loc3 = list.LocateList(4);
33         System.out.println("线性表中值为4的元素是: " + loc3);
34
35         // 测试按位查找
36         int val = list.GetList(1);
37         System.out.println("线性表中位于1号位的元素为: " + val);
38
39         // 测试删除
40         int delVal = list.ListDelete(1);
41         System.out.println("将从1号位删除的元素的值为: " + delVal);
42         System.out.println("删除此元素之后, 线性表变为: ");
43         list.PrintList();
44
45         // 测试销毁
46         list.DestroyList();
47         System.out.println("销毁表之后, 是否为空? " + list.Empty());
48     }
49 }

```

四、运行结果展示

(一) 顺序存储实现线性表

```

初始的线性表状态：
线性表表是否为空？ true
线性表长为： 0
插入元素之后，线性表长为： 11
线性表中的元素为：
1 2 3 4 5 6 7 8 9 1 1
值为5的元素的位置为： [4]
值为1的元素的位置为： [0, 9, 10]
值为15的元素的位置为： []
处于7号位的元素值为： 8
查找位置无效
处于15号位的元素值为： null
在5号位后插入一个值为10的元素，线性表变为：
1 2 3 4 5 10 6 7 8 9 1 1
8号位元素的值为： 8
删去8号位之后，线性表变为：
1 2 3 4 5 10 6 7 9 1 1
线性表在销毁后是否为空： true
    
```

顺序存储实现成功。

(二) 链式存储实现线性表

```

初始化之后表是否为空？ true
此时线性表长度为： 0
在插入之后，线性表为：
1 1 2 3 2
线性表长度为： 5
线性表中值为2的元素是：[2, 4]
线性表中值为1的元素是：[0, 1]
线性表中值为3的元素是：[3]
线性表中值为4的元素是：[]
线性表中位于1号位的元素为：1
将要从1号位删除的元素的值为：1
删除此元素之后，线性表变为：
1 2 3 2
销毁表之后，是否为空？ true
    
```

链式存储实现成功。

五、总结与收获

InitList(&L): 初始化线性表。在顺序存储和链式存储中都需要这个操作，但实现差异很大。在**顺序存储**中，需要分配一定大小的**连续内存空间**。而在**链式存储**中，只需创建一个指向 null 的头节点，使用**离散内存空间**即可。

DestroyList(&L): 销毁线性表。在**顺序存储**中，需要**彻底删除数组**，释放分配给它的所有内存。在**链式存储**中，需要**遍历链表并逐个删除节点**。

Length(L): 返回线性表长度。在顺序和链式存储中，这通常涉及到在实现数据结构时维护一个 size 或 length 标记。对于数组，可以直接返回 size；对于链表，需要遍历列表以计算它的长度。

Empty(L): 判断是否为空表。在两者均可以通过检查长度是否为 0 来实现。

PrintList(L): 输出表内容。在顺序和链式存储中，都可以通过循环来访问和打印所有元素，但在链式存储中，我们需要遵循链表的链接来访问每个元素。

LocateList(L, e): 在链表中查找元素 e 的位置。在**顺序存储**中，我们可以直接**遍历数组**来查找元素；在**链式存储**中，我们需要从头节点开始，**遍历**每个节点。

GetList(L, i): 查找特定位置的元素。使用**顺序存储**，可以**直接使用索引**获得元素。在**链式存储**中，需要从头节点开始，**遍历**到第 i 个节点。

ListInsert(&L, i, e): 在特定位置插入元素。在**顺序存储**中，需要**移动位置 i 之后的所有元素**以为新元素腾出空间；在**链式存储**中，我们只需更改一些引用即可，但需要遍历到第 i-1 个节点。

ListDelete(&L, i, &e): 删除特定位置的元素。在**顺序存储**中，需要**移动位置 i 之后的所有元素**来填补空出的空位；在**链式存储**中，我们只需更改一些引用即可，但需要遍历到第 i-1 个节点。

结论：

顺序存储（数组）通常具有快速访问元素的优势，但在中间插入或删除元素时需要移动大量元素。而链式存储（链表）在插入和删除操作上更高效，但访问特定位置的元素需要更多时间，因为需要从头节点开始遍历。

任务 2：栈

子任务一：

一、题目：两种解决 Hanoi 塔问题的方法

众所周知，栈虽然是一个操作受限的线性表，但是其用途却很广泛，栈的数据结构实现非常简单，因此我们只从应用层面熟悉栈。请完成下面三个子任务。

①递归是一种解决很多复杂问题最简单的思想方法，而任何编程语言对递归程序的支持都是通过栈实现的。请自行查阅资料理解“Hanoi 塔”问题，通过递归编程的方式解决该问题，并尝试使用非递归的编程方法解决该问题。

二、算法设计

（一）递归方法

可以将 n 个盘子看作只有两个盘子：最底下的一个盘子（最大的）和上面的 $n-1$ 个盘子（看作一个整体）。对于这两个“盘子”，我们完全可以按照汉诺塔的解决方式：

1、将上面的盘子（即 $n-1$ 个盘子看作的那一个）从 A 柱移动到 B 柱，过程中使用 C 柱作为缓冲。

2、将剩下的那个盘子（也就是编号为 n 的盘子）直接从 A 柱移动到 C 柱。

3、将 B 柱的 $n-1$ 个盘子移动到 C 柱，过程中使用 A 柱作为缓冲。

然后对于步骤 1 和步骤 3，递归地执行以上步骤，直到每一步的盘子数量为 1，也就是我们的递归结束条件。

在编写程序中，我们用三个栈 A, B, C 来模拟三个柱子，编写对应盘子的移动函数 `move`，使用递归的方式，在每一次调用递归函数 `move` 的时候都输出盘子移动的信息。

（二）非递归方法

n 个盘子的汉诺塔的非递归算法：

1. 将三根柱子按顺序排成品字型，若 n 为偶数，按顺时针方向依次摆放 A、B、C；若 n 为奇数，按顺时针方向依次摆放 A、C、B。

2. 把圆盘 1 从现在的柱子移动到顺时针方向的下一根柱。

3. 接着，把另外两根柱上可以移动的圆盘移动到新的柱上（事实上只有唯一的选择）。

4. 如果没有达到目标要求，则返回步骤 2。

三、主干代码说明

（一）递归方法 HanoiTower

```
1 package task2;
2
3 import java.util.Stack;
4
5 public class HanoiTower {
6
7     // 定义三个栈，分别表示三个塔
7     3个用法
8     Stack<Integer> A = new Stack<Integer>();
8     2个用法
9     Stack<Integer> B = new Stack<Integer>();
9     2个用法
10    Stack<Integer> C = new Stack<Integer>();
11
```

```
12 // 移动盘子的方法
13 // 3个用法
14 public void move(int n, String from, String buffer, String to) {
15     if (n <= 0) // 如果没有盘子需要移动, 则直接返回
16         return;
17     move(n-1, from, to, buffer); // 把上面的 n-1 个盘子, 借助 to, 移动到 buffer 上
18     moveTop(from, to); // 把最后一个盘子从 from 移动到 to
19     move(n-1, buffer, from, to); // 最后把 buffer 上的 n-1 个盘子, 借助 from, 移动到 to 上
20 }
21
22 // 移动栈顶盘子的方法
23 // 1个用法
24 private void moveTop(String from, String to) {
25     int disc = 0;
26     if (from.equals("A")) // 如果应该从A塔取盘子
27         disc = A.pop(); // 从A塔的顶部取出盘子
28     else if (from.equals("B")) // 如果应该从B塔取盘子
29         disc = B.pop(); // 从B塔的顶部取出盘子
30     else if (from.equals("C")) // 如果应该从C塔取盘子
31         disc = C.pop(); // 从C塔的顶部取出盘子
32
33     if (to.equals("A")) // 如果盘子应该放到A塔
34         A.push(disc); // 把盘子放到A塔的顶部
35     else if (to.equals("B")) // 如果盘子应该放到B塔
36         B.push(disc); // 把盘子放到B塔的顶部
37     else if (to.equals("C")) // 如果盘子应该放到C塔
38         C.push(disc); // 把盘子放到C塔的顶部
39
40     System.out.println(disc + "号盘, " + from + "-->" + to); // 打印盘子移动信息
41 }
42
43 // 汉诺塔的主方法
44 // 1个用法
45 public void hanoi(int n) {
46     for(int i = n; i >= 1; i--) {
47         // 把所有的盘子按大小顺序 (从小到大) 放入A塔
48         A.push(i);
49     }
50
51     move(n, from: "A", buffer: "B", to: "C"); // 通过递归移动把A塔盘子移动到C塔
52 }
53
54 // 测试方法
55 public static void main(String[] args) {
56     HanoiTower h = new HanoiTower(); // 实例化HanoiTower类
57     h.hanoi(4); // 调用hanoi方法解决3个盘子的汉诺塔问题
58 }
```

(二) 非递归方法 HanoiTowerNonRecursive

```
1 package task2;
2
3 import java.util.Stack;
4
5 public class HanoiTowerNonRecursive {
6     // 定义从 '0', 'A', 'B', 'C' 对应的字符数组
7     // 4个用法
8     private static char[] s = {'0', 'A', 'B', 'C'};
9     // 创建一个堆栈数组来存放盘子
10    // 14个用法
11    private static Stack<Integer>[] a = new Stack[4];
```

```

29 // 汉诺塔解决方法
30 // 1个用法
31 private static void solveHanoiTower(int n) {
32     // 依次将n个盘子从大到小压入第一个塔
33     for (int i = n; i > 0; i--)
34         a[1].push(i);
35     // 当盘子数为奇数时, 调整初始的塔的顺序
36     if (n % 2 == 1) {
37         s[2] = 'C';
38         s[3] = 'B';
39     }
40     // 当所有的盘子都移动到目标塔时, 结束循环
41     while (true) {
42         int next = 0;
43         // 找到最小盘子并移到下一个塔
44         for (int i = 1; i <= 3; i++)
45             if (!a[i].isEmpty() && a[i].peek() == 1) {
46                 next = i == 3 ? 1 : i + 1;
47                 move(i, next);
48                 break;
49             }
50         // 当所有盘子都移到了目标塔, 则跳出循环
51         if (a[2].size() == n || a[3].size() == n)
52             break;
53     }

```

```

54 // 记录非当前的两个塔
55 int other1 = 0, other2 = 0;
56 switch (next) {
57     case 1: {
58         other1 = 2;
59         other2 = 3;
60         break;
61     }
62     case 2: {
63         other1 = 3;
64         other2 = 1;
65         break;
66     }
67     case 3: {
68         other1 = 1;
69         other2 = 2;
70         break;
71     }
72 }
73
74 // 移动非当前塔中较小的盘子到较大的塔
75 if (a[other1].isEmpty())
76     move(other2, other1);
77 else if (a[other2].isEmpty())
78     move(other1, other2);
79 else {
80     if (a[other1].peek() < a[other2].peek())
81         move(other1, other2);
82     else move(other2, other1);
83 }
84 }
85 }
86 }

```

四、运行结果展示

（一）递归方法

用递归的方式解决汉诺塔问题：

```
1号盘：A-->B
2号盘：A-->C
1号盘：B-->C
3号盘：A-->B
1号盘：C-->A
2号盘：C-->B
1号盘：A-->B
4号盘：A-->C
1号盘：B-->C
2号盘：B-->A
1号盘：C-->A
3号盘：B-->C
1号盘：A-->B
2号盘：A-->C
1号盘：B-->C
```

以 4 为例，说明递归方法成功

（二）非递归方法

非递归方法解决汉诺塔问题：

```
1号盘：A --> B
2号盘：A --> C
1号盘：B --> C
3号盘：A --> B
1号盘：C --> A
2号盘：C --> B
1号盘：A --> B
4号盘：A --> C
1号盘：B --> C
2号盘：B --> A
1号盘：C --> A
3号盘：B --> C
1号盘：A --> B
2号盘：A --> C
1号盘：B --> C
```

以 4 为例，说明递归方法成功

五、总结与收获

递归方法是最直观的解决办法，它直接模拟了汉诺塔问题的规则；非递归的方法则是用显式的数据结构栈来保存状态信息。用非递归的方法来实现会使程序变得复杂，但是，非递归的方法对于程序的调用栈没有依赖，因此在解决大规模问题时会有更好的性能。

在实现这个任务后，我认识到，在处理可递归问题时，我们可以尝试将问题的状态转化为递归程序，然后用非递归的方式去改编，因为非递归的方法更适合解决大规模问题，这种思想在未来的学习、工作中面对更大规模的问题时都会非常实用。

子任务二：

一、题目：非递归快速排序

②请在①的基础上进一步学习掌握递归问题的非递归转化方法，并实现对递归快速排序的非递归转化。

二、数据结构设计

数据结构是一个可比较元素的数组 `Comparable[] objs`。这种结构允许算法处理各种类型的数据，只要这些数据可以进行比较操作。

本题程序用一个内部类 `Range` 来记录当前待排序的序列的起始和终止位置。在每一次迭代中，从堆栈中 `pop` 出一个 `Range` 对象，这个 `Range` 对象表示当前待处理和待排序的序列。

三、算法设计

本实验的核心是将递归快速排序算法转化为非递归形式。

每经过一次遍历，就将 `Range` 中的序列分为两个部分，其中一部分的所有元素都不大于另一部分的所有元素，并得到分区点 `pivot`。`[pivot - 1, low]` 与 `[pivot + 1, high]` 这两个子区间将会被创建为新的 `Range` 对象，然后被 `push` 到堆栈中。堆栈保证了最小的子区间首先被处理，这样可以尽快地释放空间。

这就说明可以通过使用栈（`Stack<Range>`）来模拟递归调用栈的方式实现。栈用于存储待排序的数组范围，模拟了递归过程中的调用和返回。

四、主干代码说明

（一）QuickSortOfRecursion 以及测试程序（主程序）

```
1 package task2;
2
3 import java.util.Stack;
4 public class QuickSortOfRecursion extends SortAlgorithm {
5
6     // 覆盖父类的sort方法，输入一个可比较对象的数组
7     // 3个用法
8     @Override
9     public void sort(Comparable[] objs) {
10         // 创建一个范围的栈
11         Stack<Range> stack = new Stack<>();
12         // 将数组的索引范围入栈，索引开始为0，结束为数组长度-1
13         stack.push(new Range( low: 0, high: objs.length - 1));
14
15         // 当栈不为空时执行循环
16         while (!stack.isEmpty()) {
17             // 弹出栈顶的范围项
18             Range range = stack.pop();
19
20             // 在索引起始值小于终止值时
21             if(range.low < range.high){
22                 // 对数组范围内的元素进行分区，并返回新的基准点索引
23                 int pivot = partition(objs, range.low, range.high);
24                 // 将基准点左侧的范围入栈
25                 stack.push(new Range(range.low, high: pivot - 1));
26                 // 将基准点右侧的范围入栈
27                 stack.push(new Range( low: pivot + 1, range.high));
28             }
29         }
30     }
31 }
```

```

31 // 将数组的一部分以对象高位为划分点分为两部分，返回分区后的基准点索引
    1个用法
32 @ private int partition(Comparable[] objs,int low,int high){
33     // 将最后一个对象作为基准点
34     Comparable pivot=objs[high];
35     int i=low-1;
36     // 从索引 low 开始至 high-1结束，针对数组每个元素
37     for (int j = low; j < high; j++){
38         // 若当前元素小于等于基准点值
39         if(less(objs[j], pivot) || equals(objs[j], pivot)){
40             i++;
41             // 交换 i 和 j 位置的元素
42             exchange(objs, i, j);
43         }
44     }
45     // 交换 i+1 和 high 位置的元素
46     exchange(objs, i + 1, high);
47     return i + 1;
48 }
49
50 // 定义Range静态内部类，保存索引范围
    5个用法
51 static class Range{
52     // low, high. 分别代表底部和顶部
53     4个用法
54     int low,high;
55     // 构造函数，用于新建一个范围
56     3个用法
57     public Range(int low,int high){
58         this.high = high;
59         this.low = low;
60     }
61 }
62 // 测试
63 public static void main(String[] args) {
64     QuickSortOfRecursion sorter = new QuickSortOfRecursion();
65     Comparable[] arr = {12, -7, 26, 5, -14, 30, 21, -9, 10, -28, 15, 23, -6, 19, -4};
66
67     // 打印原始数组
68     System.out.println("原始数组为: "+[12, -7, 26, 5, -14, 30, 21, -9, 10, -28, 15, 23, -6, 19, -4]);
69     // 使用sorter对象对数组进行排序
70     sorter.sort(arr);
71     // 打印排序后的数组
72     System.out.println("排序后的数组为: ");
73     sorter.show(arr);
74 }
75 }

```

(二) SortAlgorithm 类

```

1 package task2;
2
3 3个用法 1个继承者
4 @ public abstract class SortAlgorithm {
5     3个用法 1个实现
6     public abstract void sort(Comparable[] objs);
7     2个用法
8     protected void exchange(Comparable[] numbers, int i, int j){
9         Comparable temp;
10        temp = numbers[i];
11        numbers[i] = numbers[j];
12        numbers[j] = temp;
13    }
14    1个用法
15    protected boolean less(Comparable one, Comparable other){
16        return one.compareTo(other) < 0;
17    }
18    1个用法
19    protected boolean equals(Comparable one,Comparable other){
20        return one.compareTo(other) == 0;
21    }
22 }

```

```
2 个用法
17 @
18     protected void show(Comparable[] numbers){
19         int N = numbers.length;
20         int line = 0;
21         for(int i = 0; i < N; i++){
22             System.out.printf("%s ", numbers[i]);
23             line++;
24             if(line % 20 == 0) System.out.println();
25         }
26         System.out.println();
27 }
0 个用法
27 @
28     protected boolean isSorted(Comparable[] numbers){
29         int N = numbers.length;
30         for(int i = 0; i < N-1; i++)
31             if(numbers[i].compareTo(numbers[i+1]) > 0) return false;
32         return true;
33     }
34 }
```

五、运行结果展示

```
原始数组为: [12, -7, 26, 5, -14, 30, 21, -9, 10, -28, 15, 23, -6, 19, -4]
排序后的数组为:
-28 -14 -9 -7 -6 -4 5 10 12 15 19 21 23 26 30
```

说明非递归的快速排序算法完全正确

六、总结与收获

通过对子任务二的学习,我更深入地理解了快速排序的工作原理和其非递归版本的实现,同时也学习到了如何使用栈数据结构解决问题的技巧。

这个例子也证明了,通常,递归算法总是可以转换为使用栈的非递归算法,虽然在大多数情况下,递归方法更简洁,但是当递归深度非常大时,使用非递归的方法可以避免栈溢出。对于快速排序算法,非递归形式在整个排序过程中,通过交换元素的方式在原地进行排序,这也节省了额外的存储空间。

在这个过程中,我也加深了对快速排序算法理论的理解,也锻炼了自己的编程能力。

子任务三：

一、题目：双端顺序栈

③当我们在使用很多软件时都有类似“undo”功能，比如 Web 浏览器的回退功能、文本编辑器的撤销编辑功能。这些功能都可以使用 Stack 简单实现，但是在现实中浏览器的回退功能也好，编辑器的撤销功能也好，都有一定的数量限制。因此我们需要的不是一个普通的 Stack 数据结构，而是一个空间有限制的 Stack，虽然空间有限，但这样的 Stack 在入栈时从不会溢出，因为它会采用将最久远的记录丢掉的方式让新元素入栈，也就是说总是按照规定的数量要求保持最近的历史操作。比如栈的空间是 5，当 a\b\c\d\e 入栈之后，如果继续让元素 f 入栈，那么栈中的元素将是 b\c\d\e\f。请设计一个满足上面要求的 LeakyStack 数据结构，要求该数据结构的每一个操作的时间复杂度在最坏情形下都必须满足 $O(1)$ 。

二、数据结构设计

本实验的数据设计涉及到的主要是对有限大小的栈（LeakyStack）的实现。

LeakyStack 可以简单地用一个定长环形数组来实现。

通过这种设计，我们可以在常数时间内完成插入新元素和删除旧元素的操作，满足了题目的 $O(1)$ 时间复杂度的要求。此外，我们用构造函数中的 capacity 参数定义栈的固定容量。

三、算法设计

LeakyStack 的关键算法设计在于它的入栈（push）和出栈（pop）操作。

入栈（push）操作：在添加新元素时，首先更新的栈顶位置 top（通过模运算实现环形结构），然后在这个位置放置新元素；为了实现自堆底向上输出，还使用了 front 来标记。

出栈（pop）操作：返回并移除栈顶元素，同时更新栈顶位置。

四、主干代码说明

（一）LeakyStack 类

```
1 package task2;
2
3 6 个用法
4 public class LeakyStack <T>{
5     7 个用法
6     private T[] stack;
7     16 个用法
8     private int top;
9     8 个用法
10    private int capacity;
11    9 个用法
12    private int front;
13
14    //构造函数
15    4 个用法
16    public LeakyStack(int capacity){
17        this.capacity = capacity;
18        stack = (T[]) new Object[capacity];
19        this.top = -1;
20        this.front = -1;
21    }
22    0 个用法
23    public LeakyStack(){
24        this(capacity: 5);
25    }
26 }
```

```

21 //入栈
22 16个用法
23 public void push(T element){
24     if ((top + 1) % capacity == front)
25         front = (front + 1) % capacity;
26
27     top = (top + 1) % capacity;
28     stack[top] = element;
29
30     if (front == -1)
31         front++;
32 }
33
34 //出栈
35 0个用法
36 public T pop(){
37     if(top == -1){
38         System.out.println("Stack is Empty");
39         return null;
40     }
41     T temp = stack[top];
42     top = (top - 1 + capacity) % capacity;
43     return temp;
44 }
45
46 // 查看栈顶元素
47 1个用法
48 public T peek() {
49     if (top == -1) {
50         throw new IllegalStateException("Stack is empty");
51     }
52
53     return stack[top];
54 }
55
56 //判断栈空
57 0个用法
58 public boolean isEmpty(){
59     return top == -1;
60 }
61
62 //判断栈满
63 0个用法
64 public boolean isFull(){
65     return (top + 1) % capacity == 0;
66 }
67
68 //显示栈内元素
69 6个用法
70 public void showStack(){
71     if (front > top) {
72         for (int i = front; i < capacity; i++)
73             System.out.print(stack[i] + " ");
74         for(int i = 0; i <= top; i++){
75             System.out.print(stack[i]+ " ");
76         }
77         System.out.println();
78     } else {
79         for (int i = front; i <= top; i++)
80             System.out.print(stack[i] + " ");
81         System.out.println();
82     }
83 }
84
85 }

```

(二) 测试类

```
1 package task2;
2
3 public class Test {
4     public static void main(String[] args) {
5         // 创建一个字符串类型的 LeakyStack
6         LeakyStack<String> stringStack = new LeakyStack<>( capacity: 3);
7         stringStack.push( element: "Hello");
8         stringStack.push( element: "World");
9         stringStack.push( element: "!");
10        System.out.print("目前栈中元素为: ");
11        stringStack.showStack();
12
13        System.out.println("顶端元素: " + stringStack.peek());
14        // 应该显示 "!" 75. 76. // 入栈新元素, 这将导致最早的元素被移除
15        stringStack.push( element: "Java");
16        System.out.print("压入'Java'之后: ");
17        stringStack.showStack();
18        System.out.println();
19        // 应该移除并返回 "World" 81. 82. System.out.println();
20
21        // 创建一个整数类型的 LeakyStack
22        LeakyStack<Integer> integerStack = new LeakyStack<>( capacity: 5);
23        integerStack.push( element: 1);
24        integerStack.push( element: 2);
25        integerStack.push( element: 3);
26        integerStack.push( element: 4);
27        integerStack.push( element: 5);
28        System.out.println("原始数据:");
29        integerStack.showStack();
30        integerStack.push( element: 6);
31        System.out.println("操作之后:");
32        integerStack.showStack();
33        System.out.println();
34
35        // 创建一个整数类型的 LeakyStack
36        LeakyStack<Character> charStack = new LeakyStack<>( capacity: 5);
37        charStack.push( element: 'a');
38        charStack.push( element: 'b');
39        charStack.push( element: 'c');
40        charStack.push( element: 'd');
41        charStack.push( element: 'e');
42        System.out.println("原始数据:");
43        charStack.showStack();
44        charStack.push( element: 'f');
45        System.out.println("操作之后:");
46        charStack.showStack();
47    }
48 }
```

五、运行结果展示

```

目前栈中元素为: Hello World !
顶端元素: !
压入'Java'之后: World ! Java

原始数据:
1 2 3 4 5
操作之后:
2 3 4 5 6

原始数据:
a b c d e
操作之后:
b c d e f
    
```

Leakystack 类实现成功

六、时间复杂度分析

由于本题的栈是使用数组实现的，因此对任意位置的读取操作时间复杂度都是 $O(1)$ ，我们开始对每个函数都进行检验：

（一）入栈

此操作首先更新 `top`、`front` 的值，这是通过取模运算实现的，是常数时间操作。

然后，将新元素放入计算出的位置。由于数组索引是直接访问的，这也是 $O(1)$ 操作。

（二）出栈

检查栈是否为空是常数时间操作。

访问栈顶元素并更新 `top` 指针也是 $O(1)$ 操作。

（三）查看栈顶元素

仅查看栈顶元素，不涉及任何循环或复杂操作，是 $O(1)$ 操作。

（四）判断栈空

此操作仅涉及检查 `top` 的值，是常数时间操作，也是 $O(1)$ 操作。

（五）判断栈满

这个方法检查栈是否已满，同样只涉及常数时间的计算，因此是 $O(1)$ 操作。

七、总结与收获

在实现 `LeakyStack` 类的过程中，我学会了如何设计并实现一个具有特定限制的数据结构，而且也加深了对环形数组与栈数据结构的理解。这种 `LeakyStack` 的设计在许多实际的应用中都非常有用。比如浏览器的回退功能和编辑器的撤销功能，在这些情境下常见的需求就是保持最近的操作，而旧的记录会被丢弃。

此外，这个实验还强调了在设计数据结构时考虑时间复杂度的重要性，确保每个操作都能在 $O(1)$ 时间内完成。通过这个任务，我学习和理解了如何设计和实现一个满足特定需求的数据结构，这对于提高我的算法设计和编程技能非常有帮助。

任务 3：基数排序

一、题目

使用自定义的队列数据结构实现对某一个数据序列的排序(采用基数排序)，其中对待排序数据有如下的要求：

- ①当数据序列是整数类型的数据的时候，数据序列中每个数据的位数不要求等宽，比如：1、21、12、322、44、123、2312、765、56
- ②当数据序列是字符串类型的数据的时候，数据序列中每个字符串都是等宽的，比如："abc", "bde", "fad", "abd", "bef", "fdd", "abe"

注：radixsort1.txt 和 radixsort2.txt 是为上面两个数据序列提供的测试数据。

二、数据结构设计

主要数据结构：

自定义队列（Queue）：使用链表实现的队列，支持基本操作如入队（enqueue）和出队（dequeue）。

基数排序数组：整数数组和字符串数组，用于应用基数排序。

辅助数据结构：

桶（buckets）：在基数排序中，使用一个队列数组作为桶来存放特定位的数字或字符。

三、算法设计

整数排序：

根据每个数字的位数（从个位到最大位数），将数字放入相应的 0-9 桶中，然后按顺序收集这些桶中的数字。

字符串排序：

类似地，根据字符串的每个字符（从最后一个字符到第一个字符），将字符串放入相应的字母桶中，然后按顺序收集。

文件读取和写入：

读取：从文本文件读取整数或字符串序列。

写入：将排序后的数据写入指定文件。

四、主干代码说明

（一）链表节点类 Node

```
1 package task3;
2
3 1个用法
4 public class Node<T> {
5     1个用法
6     public T value;
7     1个用法
8     public Node next;
9
10    0个用法
11    public Node(T value) {
12        this.value = value;
13        this.next = null;
14    }
15 }
```

(二) 用链表实现队列操作

```

1 package task3;
2
3 8个用法
4 public class Queue <T>{
5     7个用法
6     Node head,tail;
7     2个用法
8     public Queue() {
9         this.head = this.tail = null;
10    }
11
12    3个用法
13    public boolean isEmpty() {
14        return head == null;
15    }
16
17    2个用法
18    public void enqueue(T value){
19        Node temp = new Node<>(value);
20        if(this.tail == null){
21            this.head = this.tail = temp;
22            return;
23        }
24        this.tail.next = temp;
25        this.tail = temp;
26    }
27
28    2个用法
29    public T dequeue(){
30        if(isEmpty()){
31            System.out.println("This Queue is Empty");
32            return null;
33        }
34        T temp= (T) head.value;
35        this.head = this.head.next;
36        if(head == null){
37            this.tail = null;
38        }
39        return temp;
40    }
41 }

```

(三) RadixSort

```

1 package task3;
2
3 import java.io.*;
4 import java.util.Scanner;
5
6 1个用法
7 public class RadixSort {
8
9     //数字数据基数排序
10    1个用法
11    @ public static void radixSortForInteger(int[] arr) {
12
13        //判断最大数字是几位数字
14        int max = Integer.MIN_VALUE;
15        for (int value : arr) {
16            if (value >= max) {
17                max = value;
18            }
19        }
20    }
21 }

```

```

18         int maxLength = Integer.toString(max).length();
19
20
21         //0~9 一个数字一个桶
22         Queue<Integer>[] buckets = new Queue[10];
23         for(int i = 0; i < buckets.length; i++)
24             buckets[i] = new Queue<Integer>();
25
26
27         //根据最大位数决定比较次数,从个位数开始比较,依次将其放入对应桶中
28         for (int i = 0, n = 1; i < maxLength; i++, n = n * 10) {
29             for (int value : arr) {
30                 //求对应位数应放入的桶
31                 int num = (value / n) % 10;
32                 buckets[num].enqueue(value);
33             }
34             int index = 0;
35             for (Queue queue : buckets) {
36                 while (!queue.isEmpty()) {
37                     arr[index] = (int) queue.dequeue();
38                     index++;
39                 }
40             }
41         }
42
43     }
44
45     //将结果输出到指定文档中
46     1个用法
47     @ public static void writeToFile1(int[] array, String filePath) throws IOException {
48         try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
49             int index=0;
50             for (int value : array) {
51                 writer.write( str: value + " ");
52                 index++;
53                 if(index==10){
54                     writer.write( str: "\n");
55                     index=0;
56                 }
57             }
58         }
59
60     1个用法
61     @ public static void writeToFile2(String[] array, String filePath) throws IOException {
62         try (BufferedWriter writer = new BufferedWriter(new FileWriter(filePath))) {
63             int index=0;
64             for (String value : array) {
65                 writer.write( str: value + " ");
66                 index++;
67                 if(index==10){
68                     writer.write( str: "\n");
69                     index=0;
70                 }
71             }
72         }
73     }

```

```

74     //字符串数据基数排序
75     1个用法
76     @ public static void radixSortForString(String[] strings ){
77
78         //求取最大长度
79         int maxLength=strings[0].length();
80
81         //为大小写共 52 个字母创建桶
82         Queue[] temp = new Queue[52];
83         for(int i = 0; i < temp.length; i++)
84             temp[i] = new Queue<String>();
85
86         for (int i = 0; i < maxLength; i++) {
87             for (String string : strings) {
88                 char c = string.charAt(maxLength - 1 - i);
89                 int num = c >= 'a' ? c - 'a' + 26 : c - 'A';
90                 temp[num].enqueue(string);
91             }
92             int index = 0;
93             for (Queue queue : temp) {
94                 while (!queue.isEmpty()) {
95                     strings[index] = (String) queue.dequeue();
96                     index++;
97                 }
98             }
99         }
100     }

```

```

102 //两个测试函数用来测试是否已经排序成功
103 1个用法
104 @ public static boolean isSortedOfNums(int[] nums){
105     for(int i = 0; i < nums.length - 1; i++){
106         if(nums[i] > nums[i+1]){
107             return false;
108         }
109     }
110     return true;
111 }
112 1个用法
113 @ public static boolean isSortedOfStrings(String[] strings){
114     for(int i = 0; i < strings.length - 1; i++){
115         if(strings[i].compareTo(strings[i+1]) > 0){
116             return false;
117         }
118     }
119     return true;
120 }
121 //补上两个读取数据的函数
122 1个用法
123 @ public static String[] getStrings(String filename) throws FileNotFoundException {
124     Scanner in= new Scanner(new File(filename));
125     StringBuffer sb=new StringBuffer();
126     String temp=in.next();
127     sb.append(temp);
128     temp=in.next();
129     while(!temp.equals("stop")){
130         sb.append('\n');
131         sb.append(temp);
132         temp=in.next();
133     }
134     return sb.toString().split( regex: "\n");
135 }
136 1个用法
137 @ public static int[] getNums(String filename) throws FileNotFoundException {
138     Scanner in=new Scanner(new File(filename));
139     StringBuffer sb=new StringBuffer();
140     String temp=in.next();
141     sb.append(temp);
142     temp=in.next();
143     while(!temp.equals("stop")){
144         sb.append('\n');
145         sb.append(temp);
146         temp=in.next();
147     }
148     int[] nums= new int[sb.toString().split( regex: "\n").length];
149     for(int i = 0; i < nums.length; i++){
150         nums[i]=Integer.parseInt(sb.toString().split( regex: "\n")[i]);
151     }
152     return nums;
153 }
154 }

```

(四) 读取文件

```

1 package task3;
2
3 import java.io.FileNotFoundException;
4 import java.io.IOException;
5 import java.util.Arrays;
6
7 import static task3.RadixSort.*;
8
9 public class Test {
10
11     //文件地址
12     public static void main(String[] args) throws IOException {
13         int[] nums = getNums( filename: "C:\\Users\\17519\\Desktop\\数据结构与算法\\李焱辰\\Lab2\\radixSort1.txt");
14         String[] strings = getStrings( filename: "C:\\Users\\17519\\Desktop\\数据结构与算法\\李焱辰\\Lab2\\radixSort2.txt");
15         radixSortForInteger(nums);
16         writeToFile1(nums, filePath: "C:\\Users\\17519\\Desktop\\outputIntegers.txt");
17         System.out.println(isSortedOfNums(nums) ?
18             "The numbers have been sorted!" : "The numbers are not sorted!");
19         radixSortForString(strings);
20         writeToFile2(strings, filePath: "C:\\Users\\17519\\Desktop\\outputStrings.txt");
21         System.out.println(isSortedOfStrings(strings) ?
22             "The strings have been sorted!" : "The strings are not sorted!");
23     }
24 }

```

五、运行结果展示

```
The numbers have been sorted!
The strings have been sorted!
```

整数排序后：

```
0 6 10 16 18 18 21 23 27 28
33 37 37 37 41 42 43 47 48 51
56 68 69 70 85 87 91 96 97 102
105 111 112 124 126 128 133 140 142 142
142 146 147 148 149 154 165 166 169 175
175 178 188 200 200 202 207 215 217 221
229 236 236 237 255 263 264 264 265 270
271 277 279 280 282 286 289 296 298 299
308 313 314 317 319 319 322 324 326 335
339 343 343 345 354 359 360 361 362 363
363 363 368 368 377 379 384 384 386 392
395 404 413 414 414 418 419 435 436 438
439 442 444 445 447 449 457 457 463 466
468 473 476 478 483 492 497 500 514 520
528 533 535 550 552 553 554 560 563 567
574 575 575 576 577 588 590 600 603 614
615 617 618 637 638 643 643 653 654 657
657 660 664 668 671 692 695 714 720 724
731 735 736 737 743 744 746 753 753 759
764 771 783 786 792 798 798 800 802 803
805 813 814 824 829 838 843 862 865 869
874 875 877 881 882 882 887 891 891 895
900 902 908 909 909 912 913 916 936 936
937 939 944 946 947 951 953 961 964 967
967 968 972 974 975 975 977 985 985 985
990 997 998 1000 1002 1003 1017 1018 1019 1020
1031 1031 1033 1037 1041 1050 1054 1058 1063 1070
1077 1079 1082 1084 1084 1086 1087 1087 1089 1096
1097 1098 1100 1103 1103 1109 1114 1127 1130 1132
1138 1148 1155 1159 1161 1162 1164 1164 1174 1181
1185 1186 1186 1187 1189 1212 1213 1216 1219 1222
1224 1227 1228 1230 1231 1236 1239 1240 1241 1244
1274 1282 1284 1293 1295 1296 1297 1307 1309 1313
1321 1330 1332 1332 1339 1339 1342 1343 1345 1347
1349 1350 1359 1361 1363 1367 1380 1381 1381 1385
1395 1398 1404 1406 1407 1409 1412 1424 1426 1427
```

.....

```
1084509 1084546 1086065 1086809 1087383 1087683 1088488 1088631 1089003 1089956
1090201 1090972 1092024 1092189 1092692 1092927 1093078 1094687 1095673 1097685
1100058 1100687 1101311 1102232 1103190 1103453 1103650 1104039 1105720 1108402
1110509 1111005 1111639 1113346 1113425 1114002 1114106 1115015 1118243 1118535
1118981 1122375 1122440 1124109 1124240 1124401 1124560 1124960 1125876 1128908
1128976 1129114 1129244 1129577 1129869 1130717 1132250 1133015 1134215 1134542
1136467 1137253 1137502 1138252 1138498 1138808 1139237 1140000 1140217 1141194
1142686 1145847 1146680 1147051 1147200 1147608 1148092 1148364 1148910 1149127
1149481 1149689 1150093 1150142 1150798 1152640 1152970 1153389 1153483 1153878
1153943 1154110 1156861 1161139 1161462 1162076 1162342 1163524 1164479 1164499
1164561 1164718 1165166 1165534 1167366 1168040 1169896 1170951 1171260 1173378
1173408 1174101 1176164 1177414 1178092 1178337 1179383 1179499 1180574 1181343
1181623 1183479 1183684 1184024 1184391 1184917 1188641 1188971 1189185 1189582
1189753 1190630 1191226 1191454 1191564 1192946 1194653 1195465 1195705 1196882
1198179 1199314 1199819 1200850 1201068 1201257 1202775 1203380 1203540 1203580
1203599 1204609 1207368 1207500 1207957 1210087 1210693 1211032 1213413 1214466
1214714 1214914 1216408 1217347 1218801 1219176 1219773 1220457 1221077 1221546
1221558 1221831 1222154 1222410 1222540 1222578 1223702 1224240 1224891 1225525
1225955 1226036 1227416 1228735 1228800 1230213 1230231 1231992 1233870 1234264
```

字符串排序后:

```
ACgnsNit ADBJMSzB ADRKabPw ADRKsb1B ADjSFRsM AEppMxnC AGIWmdlX AKpTKsKE ALPQRdWg AP0eBcfB
AQdwNrjL ARzprbYP AUNkreFd AwtdOqOs AXQvcSbs AVOGHDOX AYbuimYA AYvhbcNk AanVzwtS AgYNUvcd
AlRpOQFn AlOcZkRl AmpCpEzH ApyjWzTA AqcsZvdQ AsasKDXt AuAkCOZp AyeMQadX AzbxOQJz AztMmxSg
BB0JEHQ0 BCKKRHgK BCVEDsin BEeMSeCF BFPClXne BFsZzJdJ BGBPBCFy BHXvhnwU BIAgsszZ B1pcyQtC
BKthFwAh BLSapdFF BLpdDeyE BLrxGkDU BMSgeojg BNDToivF BNoQwkio BPEewCuy BQLpANqw BRdegocn
BRvuBbJk BSvePbmL BVowzoaV BXsNOPsc BYExhMvV BZROZehw BZWGzKSb BZatoZIY BcfSsG1I BdldiaVx
BeESGied BeNiNNeP BeYlGzDR BeBCMCFG BFQXQrSq BhdMwoqp BjIpWzKO BlMgPzth BldNMuoT BluZbTNF
BmA0dcVg BmwJuUBP BnJDKKnP BowLatbX BoqxerFF BpLktyNI BqUXvyKK BrOWZIQJ Br1jOWPU BrwMILAt
BsMzLWgB BsOLVcXt BTIZCAIO BtPvaogP BwKtXaWi BzwqTzbP CBRRnnMj CHYPClDu CKFFxely CKzDQoKU
CNTVdFNQ COFaEqsA CQXrSMYS CSNLVWRY CwjPhsZI CZGygfyD CbjEjRiS CbkthTGB ChVrQQNP ChYqdePX
CpUMDokm CsOLlPXE CsReToVT CtyoeBif CuYqXlxc CvVFDJGp CxwtNrcj CzUcMLbQ CzYvaaoj DAieCqpw
DAwHxUII DECHGgPr DERGcsXW DGEJHqLJ DJAlTpBp DJVEqGKh DNEHJNNH DOIzzAIY DQpJSjFZ DSDJEFT1
DTGETGoI DUmmGmkV DVIUADcP DVrdTWAG DYYJZWot DZpvQiND DcuwFuDM DdXkQqWb DdbKZMDY DeQbsPHB
DfiAuKcs DhBCTDdg DhJkthfw DhXNmMR D1bXlSnC DlnBLKaO DmRlOlwm DnQENUKB DoMuQsvu DoPlQappt
DQojhoSA DotyEsrg DqMuGCEr DqfgvmPY DrOcfQgB DSAuykiF DvTjTKqs DxoBFntt DyFEruoL EbfyRheE
EBUAWzJK EBzwtuMw EDbqMzSi EDdYGNPy EdqTfbOB EFmHqVlW EHPZSHQD EIfvFvFB EnIZuTza EpJzRNKv
EKcuHzqc EOJntupu EPVRgPsZ EQLOjPpF ERPmqnsu ETLLeOJFX ETLkqTdN EUwmjXbc EVJfyFIL EVsWYzAH
EVzZBZzk EYyjGsy1 EYgwhHAW EYzBBKvY EaQGvbgP EavcnujP EbFGvKAK EbhwbXkm EdMuaVYD EeVKdyDj
EgJSSaAA EhwUEJTY EilwMEPB ElqPuFvK EmNnwXNS EmwIwMDL EmfZTCBY EnIZuTza EpJzRNKv EFCcrHYUc
ErMskxET EsCgaPrL EuiVkpSF EvUmGRDK ExQSCffk ExTgEnDT EyVBnyXu EyXXtZNf FccrHYUc FCwTVUAF
FDgoZVjN FEXMDgUb FFDsrSzo FFnKjQdk FGIOvVOW FHTidlBg FHTlpsYc FHWtfrlZ FIsIkULW FK0gQcPo
FKyEBPKm FMGZAay FMzPHWAd FOlvRYDO FOvLcbrt FPahkMue FQXBjKpQ FXGzDnhz FYbtPatY FZWZHTZD
FZYPLNzB FZeedXHF FZjCpirc FarfTEoH FcBidqdy FdyOAIWu FKQlOfbd FliabauB Flzxyxxy FzZVQoKU
FpKtIymA FqRPXuNQ FuiyaYQf FvLybtNi FvUMRDJb FvzQlMfY FxSEupcd FzPLBzSG FzYBLtbd GCAEF1Tk
GCDKlEdF GCSBZHTY GCaZWpgj GFBsWBPD GIQTmyzf GJDFwmVn GJJnbHEN GKCrVZLk GLJmJhMi GMfKdNth
GODXKHZs GOTSkgMU GRJDeiZB GRhiGRFe GSFReRbe GUUyyJLR GvCEcmFf GYskJRfK GzoUZcdp GbDmWlRQs
GbTatKSK GcGcwuhr GdOdMtZE GeQlSTiv GjLPfZHC GkBrkYom GkTanKjv Gknhihhs GmYBDNc GmioEHXJ
GoHjvKpw GocDeEwt GqQzoPeT GsLZqbvw GsQvOpmv GSxYzNg GsqYNBis GvCchkeU GvDeIXPA GveBXjaT
GwGMPTAV GwOvXkwx GxAeJGwe GxDrHrrw Gx1Kpiu GxtFTxoz GyjtdFzN HarnkdEU HDLDYAFB HDXsFOQo
HITKjvWQ HIsnaERr HLSojusj HMGRFufh HPBUhuxS HPMhBTme HQGuqCop HRdwZnvS HSuLgxxR HTTfskel
HVRgSQaT HYCxLykW HYGCVePI HYarFmDN HZKTnVPH HZUGIHGR HaAHNCEq HaxQWgji HbHesScN HbWKhIqv
HdSremou HdTbOYhD HfCZgMae HflvPYeo HgJnRiux HHuHmOmy HhkxyXgc HijKukta HioUtMXo HlaeevsN
HqOFVOuy HraqLdVw HsBcxRwD HwCvDotb HwSJcmr1 HyRiZEIz IBwtUPPi ICdJqTLW IDOOvsnx IJDRag1H
IJWvKNTy ILSQxwMW ILpRfpkF INosAxVq IPeWnUrX IPhoYJHQ IQhAvhRZ ISfNgsgo ITDwMXng IUWzXXFI
IUsmJNmV IWYDockX IYDuHJTV IYDyQiiu IYsojrwE IZDpTHfE IbfAmXhx IcgDWCoP IeIJGRsv IeQYwSxn
IfVwYlEG IibBuPXg IkFtfrdJ ImOiefnJ IoaXfJBQ IoZlztvx IrTwtgYw It1lJfXQ IwQymbJt IdXHJHJm
IxSCgfaF IxmGKRHW IyCJOHsn IyJqUcuQ Iygmduxj JAhoYuro JFXmNqxa JGERhkkB JGXBKRbf JImqPIJC
JInBshBN JJYOpAcz JKEyjkNm JKsxhcDj JNWLldtF JNoiUpib JQiqnxAm JQmrgLxR JgttvGgj JjIgbYQV
JVCzQbYy JXQSWHqT JYKibvSO JYRCrtkt JdugyOeC JeDuPySx JePJJcsB JgttvGgj JjIgbYQV JkASbZOq
JnsnmYyu JqIYooUJ JqchfsDZ JrZmDQlI JrbpqbRq JuQJPScH KABdlLVb KBTZudej KBfionGw KCYVMLWH
KCiAPHib KCrCRFjv KCuveSUF KCwIQzZn KDCLMZVm KDIftNu1 KDfdtRbK KEgiBXDm KFRJzBDe KIqHgtMh
KJTeefJQ KKECezLpg KKAxCTAX KLFznHfw KNAUOpnA KRUEapns KSCyQJnd KTtGDeEK KUQQZpDs KVsUWRKL
KZBBbVUP KZKqdilZ KZgZrDMt KaekRHwM KbwIWB1I KcLSAowQ KdIYgrNk KgaBokeB KhiLiK1k KiGrCjon
Kiwmnqgh KkdztITW KlGrckDy KoYzMNCh KptuyFbk KsIDaqCT KuvxNKdC KvZmrWEO KxwWujTk KyNcTDbb
KyPuyQGW KzJQihZu LAaCKJA LCxFoNXR LELMxJeP LERApw1j LFeZAQCa LHdJbXKG LIfvKBcc LIgKsqzC
LLAbZdJd LLPhVqi LOQWguBi LQtMFMf LRFNFSsp LsuZtcd LvvZaQtE LwbDuzi LwPNYDmQ LZREDk1K
LcnvjAaN LdSLGvY LeGQlVsY LgfmLwX LgssCyEj LiZKtGwt LjzJCWCN LmZJZmRO LmlpqFbd LqJPSzWA
LrPhCkqt LriOKwMM LtLAVSTu LvouSyGU LvttCiCL LyQHCJv1 LzjJfYmk MAMORPiQ MEwo1reb MFGSUTKr
MFOersyW MHcwvKCu MHVjiFsL MLChYbry MLRErnmE MLaOubIB MMISFRwv MnicBQtn MoyTcIQ MPgAQRLS
MTZePcsc MZgIleU MUgoEwDz MUMCIUTf MUMonznM MXmqANRY MahpYhZQ MdTdmJEh MeT1td1T Mewlynpc
MfDcfEue MgnSCSia MhZYVDaE Mh1OPeyo MIAFYFYt M1UrcRw1 MkhnRFL1 MkndsLPC MpIusXUP MphjXhFy
MqkZYvWV MsmobpTm MtXKibOE MtjQXZUu MuqISGbj MvbiGdaQ MvgYAHVY MxHMBtsQ MyCFEGyX MygtEtot
NCEycQhm NDFbvreI NDceLv1z NFzQzzvw NGaYrtGc NHIAiYUA NHsonMMf NIHXHivM NJzYSMEl NKUqhW1
```

.....

```
ybcppa yghyap ygmoe ygyrme ygyvce ygmam ygyeje ygyeqm ygmqr ygyeae
ybzBiXoT yedDKlXK yhhhUmFZ yizYwiMC ykEgUsRb ympNuhHq ynumxUjF yqBvFgfO yqNbJJvp yqRzxmzw
ysbTEeic ysualdEX ysyceUPZ ytVxNvTu yvAoheKA yvFACzzZ ywubSoqX yx1ki1hT yyQtFXvm yzthZHXm
yzuRrjUp zAP1ZGHC zAhjQveW zAlDCdbN zBXpCARs zDvJ0oRI zE1zxSXB zFKxUWQM zFzCSckv zIxEOCBQ
zJGuBHZt zKgtGunP zKjoVYwz zKohVqQR zLWHYmeQ zLXFseML zLcAlmIx zMMTWdhx zOWHMzJD zOnAJYqj
zQCRFyWz zQPClTub zRLiouoj zSNcqCDk zTDZiQuz zUwIWSML zVFWiLi zWsvWMTz zXBRmOfJ zXWmZNRr
zYNPcfoS zZfrYBW zZgkXqgt zbJNVRKU zbVBOAXn zcjHhskB zdROHPxm zdzRKwXO zeEazNEa zelaQmtc
zfrVOagj zIERTCDH zJdxuOGn zjhZTuac zjwiYEqc zthWBwXw zvrAVQ1s zwLTMxHN zyCbelhX zyknRWGz
```

六、总结与收获

这个实验提供了实践自定义数据结构（如队列）和复杂排序算法（如基数排序）的机会。通过这个题目，我们不仅学习了基数排序算法的原理和实现，还加深了对文件操作、数据结构和算法的综合应用的理解。此外，实验还提高了对不同数据类型处理的灵活性，以及如何根据数据特性选择合适的排序算法。通过这个实验，我们提升了编程技巧和解决实际问题的能力。